

RWA Zadaća 1: “Upravljanje multimedijским kolekcijama” V1.1

Poslužitelj: spider.foi.hr

Korijenska mapa zadatka: /var/www/RWA/2025/zadaca_01/{LDAP_korisnicko_ime}/

Pristup preko web preglednika: http://spider.foi.hr:{vaš_port}

Portovi: /var/www/RWA/2025/portovi.js

Baza: RWA2025{LDAP_korisnicko_ime}.sqlite

Vanjski Servis “The movie database” (TMDB): <https://developers.themoviedb.org/>

Vanjski sadržaj “YouTube”: <https://www.youtube.com>

Funkcionalni zahtjevi Web aplikacije

Web aplikacija pruža korisničko sučelje za upravljanje multimedijским kolekcijama proizvoljne tematike. Web aplikacija upravlja multimedijским sadržajima dobivenih sa TMDB servisa i vlastitim multimedijским sadržajima.

U aplikaciji postoje 4 uloge korisnika (admin > moderator > obican korisnik > gost). Svaka uloga više razine ima pristup pogledima niže razine. U tablici 1 dan je opis funkcionalnosti po ulogama s aspekta korisnika. U nekim pogledima (navedeno u tablici 1) ako se dohvaća više podataka treba implementirati stranicenje (ili neki drugi oblik prihvatljivog učitavanja sadržaja) koje radi na poslužiteljskoj strani, a klijentska strana dobiva samo podatke te stranice.

Na svim stranicama treba biti navigacija te ni jedna stranica ne smije biti “slijepa ulica”. Svaka stranica mora imati gumb za prijavu ili odjavu ovisno o statusu prijavljenosti. Ne smije se dozvoliti neovlašteni pristup stranicama. Dodavanje podataka mora proći validaciju na poslužiteljskoj strani kako ne bi bilo moguće ubaciti nepoželjne znakove i/ili sadržaj.

Na stranicama koje koriste servis treba napraviti tako da se stranica NE osvježava i da se koristi Fetch API za komunikaciju sa servisom. Može se koristiti i na drugim stranicama, ali nije obavezno.

Vizual stranice potrebno je oblikovati CSS-om, a podatke strukturirati na prikladan način (npr. Tablica, lista i sl.)

Tablica 1. Pregled funkcionalnosti Web aplikacije

Uloga	Funkcionalnost	Opis
Gost	Početna	a) Vidi pregled javnih kolekcija sa istaknutom slikom. b) Može vidjeti javne multimedijske sadržaj u kolekciji.
Gost	Dokumentacija	a) Prikazuje stranicu dokumentacija.html opisanu kasnije.
Gost	Prijava, Odjava	a) Korisnik se može prijaviti ili odjaviti. Prilikom prijave kreira se sesija dok se kod odjave ista briše. b) Ako tri puta unese pogrešna lozinka račun se blokira i administrator ga mora odblokirati prije daljneg korištenja.
Gost	Registracija	a) Gost se može samostalno registrirati kao novi korisnik. Kod dodavanja mora obavezno unjeti email, korisničko ime i lozinku. Opcionalno može unjeti ime, prezime i još barem 3 druga podatka po vlastitom odabiru. Lozinka se u bazu NE smije spremi u čitljivom obliku i treba koristiti sol kod spremanja.
Korisnik	Kolekcije	a) Vidi pregled svojih kolekcija sa istaknutom slikom. b) Može pregledati kolekciju i vidjeti multimedijske sadržaje odabrane kolekcije. c) Može promjeniti kolekciju u javnu/privatnu i promjeniti istaknutu sliku kolekcije. d) Može dodavati nove multimedijske sadržaje u kolekciju i brisati postojeće. Svaki sadržaj može biti javan ili privatan. e) Može vidjeti metapodatke sadržaja (naziv, autor, datum, veličina, tip, ...).
Korisnik	Sadržaj	a) Može pretraživati multimedijske sadržaje dohvaćene iz TMDB servisa. Upisom naziva pretražuju se zapisi TMDB servisa (npr. filmovi, serije, kompanije, ...) . b) Odabirom zapisa prikazuju se multimedijski sadržaji (slika, video). c) Mogu se zapisi dodati u kolekcije za kojima korisnik može upravljati, slika i video koji dolaze iz TMDB servisa se ne preuzimaju već se samo sprema putanja u bazu koja se koristi u prikazu. d) Može postaviti (eng. upload) novi vlasiti multimedijski sadržaj. Kod toga mora ručno dodati metapodatke ili ih učitati automatski ako je to moguće. Postavljanje sadržaja ide u poseban direktorij dok se u bazu sprema samo putanja i/ili naziv datoteke.
Moderator	Kolekcije	a) Moderator kreira kolekcije. b) Dodjeljuje korisnike u kolekciju koji mogu njome upravljati. Jedan korisnik može imati više kolekcija, a jedna kolekcija može imati više korisnika koji njome upravljaju.
Admin	Korisnici	a) Vidi popis i podatke svih korisnika. b) Može blokirati/odblokirati nekog korisnika. Ne može blokirati sam sebe. Blokirani korisnici su posebno označeni. c) Može promjeniti ulogu nekog običnog korisnika u moderatora i obratno.

Opće upute i pravila

Način predaje: Rješenje postaviti na poslužitelj do dogovorenog roka, u mapu za zadaću, a ne u mapu za vježbe. Treba predati kompletan projekt (razvojni TypeScript i izvršni JavaScript) Nakon prijena postaviti prava (-rwxrwx---) za taj direktorij (chmod 770). **Zadaca postavljena na krivo mjesto ili nakon dogovorenog roka se ne ocenjuje i rezultira s 0 bodova, a nepravilno definirana prava pristupa rezultira s oduzimanjem do 50% ostvarenih bodova.** Prava datoteka i poddirektorija unutar tog direktorija nisu bitna.

Struktura zadace: U direktoriju smiju biti samo datoteke i poddirektoriji vezane uz zadaću, sve ostalo treba ukloniti (npr. skripte s vježbi). **Svugdje u zadaci treba koristiti relativne putanje.** Ako u tekstu zadace piše naziv/struktura neke datoteke ili direktorija tada se treba držati tog naziva/strukture u ostalim slučajevima naziv/struktura je proizvoljna. **Multimedijalne datoteke postavljene na server moraju poštivati maksimalne veličine 500KB slike i 1MB video!** Ako **se ne pridržava zadane strukture direktorija, nazivlja datoteka i veličina datoteka** student se kažnjava s **oduzimanjem do 50% od ostvarenih bodova.** Poslužitelj **spider.foi.hr** prepoznaje velika i mala slova (eng. case sensitive) kod naziva direktorija, naziva datoteka i ekstenzija. Osnovna struktura je sljedeća:

- **src** - sadrži TypeScript - organizirano u smislene cjeline
- **build** - jednaka struktura kao za src samo sadrži kompajlirani JavaScript
- **dokumentacija** - direktorij sadrži html stranice dokumentacije koje poslužuje web aplikacija na putanji /dokumentacija
- **podaci** - direktorij sadrži podatke konfiguracije i bazu
 - **konfiguracija.csv** - konfiguracijska datoteka koju koriste servis i aplikacija kod pokretanja
 - **RWA2025{LDAP_korime}.sqlite** - SQLite baza podataka sa podacima servisa
- **package.json** - npm konfiguracija - koristi se za pokretanje servera sa npm start
- **tsconfig.json** - konfiguracija za TypeScript
- **package-lock.json** i **node_modules** - automatski kreirani od strane npm-a

Anketa/Dokumentacija: Potrebno je ispuniti dokumentaciju i predati zajedno sa zadaćom te anketu vezanu uz zadaću na Moodle sustavu. Ako **se ne ispuni anketa/dokumentacija ili se ne ispuni do kraja ili sadrži pogrešne informacije** oduzima se **do 50% od ostvarenih bodova.**

Napomena: Svi negativni bodovi se akumuliraju na ostale negativne bodove. Na primjer ne predana anketa, pogrešno definirana prava i pogrešna struktura mogu rezultirati s više od 50% penalizacije.

Obrane zadace: Student/ica brani zadaću na posebnom terminu konzultacija. Nastavnik određuje koji studenti/ce trebaju usmeno braniti svoju pojedinačnu i/ili završnu verziju zadace. Ukoliko student/ica odbije braniti pojedinačnu i/ili završnu verziju zadace ili ne zna objasniti rješenje gubi bodove zadace. Na obrani otvara kod zadace i objašnjava se što radi neki dio programskog koda koji nastavnik odredi. **Treba poznavati svaku liniju napisanog koda u zadaci u kontekstu programskog jezika u kojem je napisan, ali i logički koji je smisao koda u kontekstu zadace.** **Ako student/ica ne da odgovor na postavljena pitanja i time ne obrani zadaću dobiva se 0 bodova iz zadace, a time se gubi i potpis prema modelu praćenja.**

Pomoć i pitanja: Sva pitanja u vezi zadaće postavljaju se u **poseban forum za zadaću na sustavu Moodle**. Nema privatnih konzultacija u vezi zadaće osim ako nastavnik na Vaše pitanje u forumu ne kaže da dodete na konzultacije zbog specifičnosti problem. Preporučujemo da ne šaljete kod drugima, ne pokazujete kod drugima i ne radite zadaću zajedno iako ste cimeri ili se poznajete već 10 godina. **Zabranjeno je prepisivanje** zadaća; ako se utvrdi da je zadaća prepisana od drugog/ih studenta **svako dobiva 0 bodova**.

Bodovanje: Točna skala bodova nije unaprijed poznata i određuje se tek nakon što završi predaja zadaće. Boduju se dijelovi koji su rađeni samostalno nakon vježbi. U bodovanje, uz činjenicu radi li aplikacija prema specifikaciji ulazi: kvaliteta ispunjenosti ankete, kvaliteta programskog koda, model baze podataka, dizajn aplikacije, HTML/CSS validacija, količina grešaka u implementaciji, kvaliteta provjere unosa podataka od strane korisnika (sigurnost) i sl. Ako neki dio servisa ne radi po definiranoj specifikaciji dobiva se 0 bodova za taj dio, ali se mogu ostvariti bodovi za web aplikaciju koja komunicira sa takvim servisom. Pripazite na velika i mala slova, točku, zarez i sl. Dijelovi označeni **plavom pozadinom** su za dodatne bodove što znači ako se ne naprave, može se ostvariti maksimalni broj bodova iz zadaće. **Implementirani servis bez funkcionalne web aplikacije koja ga koristi ne nosi maksimalne bodove iako je ispravno implementiran.**

Kodna stranica: Sve stranice kada se učitaju moraju ispravno prikazivati hrvatske znakove.

Dizajn: Dizajn stranice je za sada proizvoljan, važno je da se stranica može normalno koristiti u preglednicima Firefox i Chrome/Chromium. U ovoj zadaći nije potrebno raditi prilagodbe za mobilne uređaje i tablete, već isključivo za pregled na računalu.

Okviri/moduli/biblioteke: U zadaći na strani klijenta i strani poslužitelja se smiju koristiti samo okviri/moduli/biblioteke obrađeni na nastavi. Konkretno za NodeJS mogu se koristiti oni moduli koji su instalirani na poslužitelju globalno i **ne smiju se instalirati dodatni moduli na razini klijenta**. Popis dostupnih modula dobiva se sa: **`npm -g list`**

JavaScript klijentska strana: Prilikom izrade zadaće potrebno se voditi principima **nenametljivog** (eng. unobtrusive) JavaScript-a (Hint: addEventListener). Cijeli JavaScript kod se nalazi u vanjskim datotekama za pripadajuće stranice u kojoj su definirane vlastite funkcije i događaji na HTML elemente (NE atributi upravljača događaja). Svi osluškivači se učitavaju nakon učitavanja cijelog HTML dokumenta.

Baza podataka: Svaki student/ica kreira svoju vlastitu SQLite bazu podataka. Kreirana i popunjena baza se mora također nalaziti u direktoriju zadaće. **Nastavnik neće bazu samostalno kreirati.**

Čisti kod: U zadaći se treba voditi principima čistog koda što ukratko znači sljedeće. Kod organizirati u funkcije, klase i module. Moduli sadrže klase, a klase sadrže funkcije koje čine smislenu cjelinu. Funkcije ne bi trebale imati više od **35 linija** programskog koda programskog koda, u što se ne broji definiranje metode, njenih argumenata i lokalnih varijabli, prazna linija, linija samo s { ili }. U jednoj liniji nalazi se jedna instrukcija. Linija nema više od **120 znaka**. Radi se uvlačenje koda s **4 znaka** space ili 1 tab. Nazivi klasa/funkcija/varijabli su smisleni i oslikavaju svrhu klase/funkcije/varijable. **Nazivi datoteka, direktorija, klasa, funkcija, varijabli i sl. pišu se na hrvatskom jeziku.** Naravno principi čistog koda koji su spomenuti na OWT za HTML i CSS također treba koristiti.

Komentari: U zadaći ne smiju biti komentari koji objašnjavaju kod. Kod koji je zakomentiran jer se ne koristi preporuka je da se obriše, ali nije problem ako negdje slučajno ostane. Zakomentiran kod također treba znati objasniti.

Datum i vrijeme: Svi datumski podaci moraju se na ekranu prikazivati i unositi u obliku dan.mjesec.godina za datum i sati:minute:sekunde u 24 satnom obliku za vrijeme. Izbjegavati input element tipa date i tipa time jer oni koriste Američki model datuma i vremena.

Provjere unosa korisnika: Preporuka je da se sav korisnički unos kontrolira kako ne bi bilo moguće srušiti aplikaciju krivim unosima ili napraviti napad. Preporuka je korištenje **regularnih izraza** jer nosi više bodova nego varijanta s if-ovima.

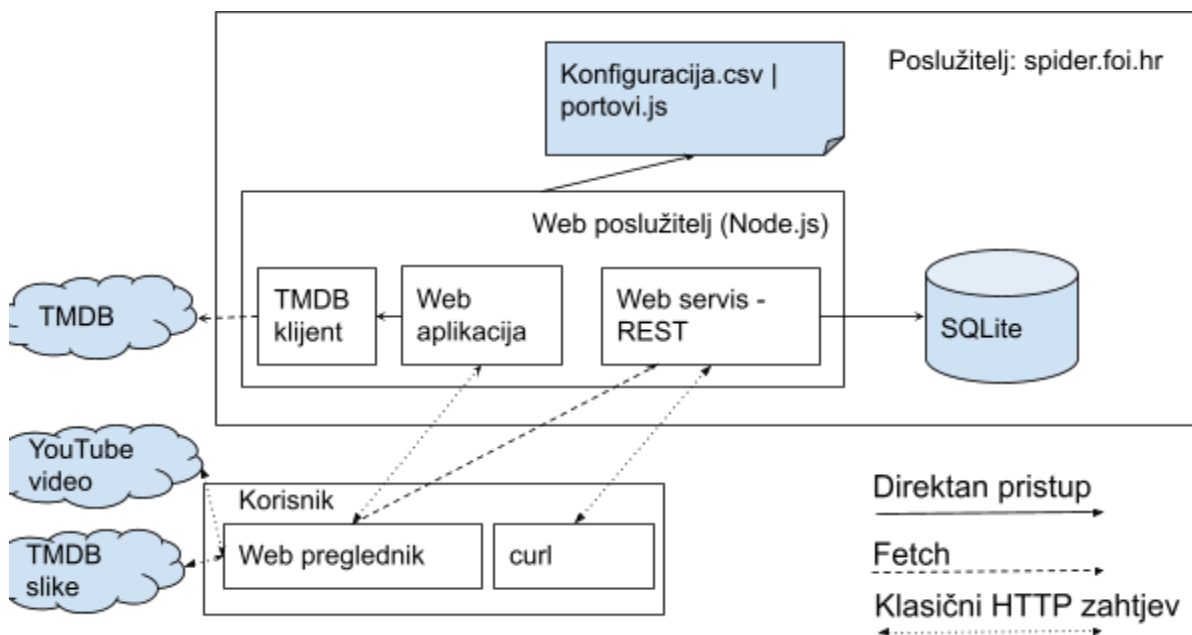
Testiranje: aplikaciju/servis treba testirati i s krivim podacima, provjeriti rubne slučajeve, a ne samo idealni scenarij korištenja. Kod testiranja servisa predlažemo **curl** naredbu jer možete lako više puta izvršiti. Također radi lakšeg testiranja bolje je napraviti prvo servisni dio, a tek onda aplikativni dio koji koristi servis.

Korisne TMDB adrese

- Registracija: <https://www.themoviedb.org/signup>
- API ključ: <https://www.themoviedb.org/settings/api>
- Dokumentacija: <https://developer.themoviedb.org/reference/intro/getting-started>
 - **Traženje:** Collection, Company, Movie, Multi, Perso i TV
 - Npr. <https://api.themoviedb.org/3/search/company>
 - **Slike:** Svaki od segmenata koji se mogu pretraživati nudi skup API-a među kojima je “images” koji traži slike (npr. slike za kompaniju)
 - Npr. https://api.themoviedb.org/3/company/{company_id}/images
 - **Video:** Filmovi i Serije omogućuju pretraživanje video zapisa sa youtube stranice čiji odgovor sadržava ključ (npr. 4aUGLJicck8) video zapisa
 - Npr. <https://www.youtube.com/watch?v=4aUGLJicck8>
- API bazični URL: <https://api.themoviedb.org/3/>
- Putanje do slika proučiti: <https://developer.themoviedb.org/docs/image-basics>
 - Npr. <https://image.tmdb.org/t/p/original/cVbBoKxRDFOdDKwdpRmxVazDWIE.jpg>

Globalna arhitektura i Dokumentacija

Zadaća se sastoji od web poslužitelja (servera) čija globalna arhitektura izgleda kao na slici 1 i mora se obavezno poštivati uključujući veze između pojedinih komponenata. Web poslužitelj se obavezno pokreće na vama dodijeljenom portu. U realizaciji je poželjno primjenjivati asinkrono programiranje, a obavezno ni u jednom trenutku ne bi trebalo doći do blokiranja glavnog programa ili “rušenja” poslužitelja. Poslužiteljski dio web aplikacije potrebno je implementirati u TypeScript-u i kompilirati u JavaScript. Klijentska strana web aplikacije je čisti JavaScript.



Slika 1. Globalna arhitektura sustava

dokumentacija.html

- sadrži informacije o Vama kao autoru: slika lica kao na X-ici, ime, prezime, FOI e-mail.
- sadrži slike ERA modela u manjoj veličini, a pritiskom na sliku otvara se originalna slika koja je kreirana s alatom MySQL Workbench
- sadrži tablicu specifikacije implementacije servisa (opisan kasnije)
- sadrži tablicu 1 ovog dokumenta, ali umjesto opisa piše da li je nešto napravljeno ili ne po svim točkama
- sadrži dodatne informacije koji dijelovi nisu napravljeni, dodatne informacije o greškama ili dijelove koji se žele posebno istaknuti

Baza podataka

Bazu podataka je potrebno samostalno modelirati i kreirati ERA model baze u alatu MySQL Workbench. Baza obavezno sadrži podatke o korisnicima sustava (slika 2). Podatke tablice “korisnik” treba proširiti sa dodatnim atributima po potrebi. **Baza sadrži sve ostale podatke i pristupa joj direktno jedino web servis, web aplikacija nema pristup bazi podataka.** Potrebno je kreirati poseban direktorij **dokumentacija** u kojem se nalazi:

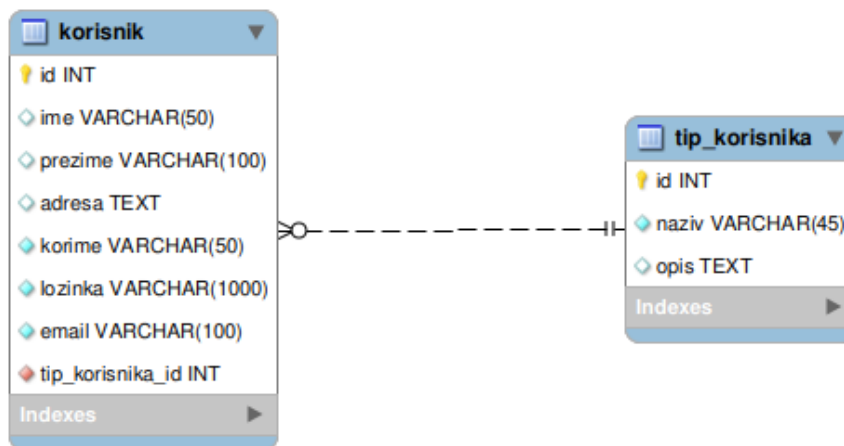
- datoteke modela baze iz alata MySQL Workbench,
- **baza.sql** koja sadrže SQL kod za kreiranje SQLite baza,
- slike ERA modela baza koji se izvoze iz alata MySQL Workbench i

Slike ERA modela morju biti dovoljne veličine da se sve vidi čitati, mora imati jasno vidljive veze (identificirajuće/neidetificirajuće), kardinalnost i svi atributi u tablici. Ove slike **SMIJU** biti veće od 1MB. U bazi podataka mora se nalaziti minimalno 10 podataka ukupno za sve glavne entitete (npr. korisnik).

Za potrebe testiranja u bazi moraju postojati tri korisnika.

- **Prvi korisnik ima korisničko ime “obican”, ulogu “registrirani korisnik” i lozinku “rwa”.**
- **Drugi korisnik ima korisničko ime “moderator”, ulogu “moderator” i lozinku “rwa”.**
- **Treci korisnik ima korisničko ime “admin”, ulogu “administrator” i lozinku “rwa”.**
- **Ako korisnici ne postoje u bazi, neće se bodovati dio koji ulazi u funkcionalnosti odgovarajuće uloge.**

Kod modeliranja baze pročitajte cijeli tekst oba dijela web aplikacije i web servisa da biste dobili sve entitete i attribute koji Vam trebaju. Vaš servis nužno ne diktira kako izgleda baza i možete u bazi imati podatke na jedan način posložene, a servis ih nudi na drugi način.



Slika 2. ERA model baze korisnika

Opis rada web poslužitelja

Kod pokretanja poslužiteljskog programa web servisa ili web aplikacije koristi se:

npm run zadaca nazivDatotekeKonfiguracije.conf [port]

Prilikom pokretanja poslužitelja provjerava se dali je proslijeđen jedan ulazni parametar ili dva. Nakon toga provjerava se da li postoji datoteka konfiguracije i dali su podaci u datoteci ispravni. Ako datoteka konfiguracije ne postoji, javlja se greška i poslužitelj prestaje s radom. Ako konfiguracijska datoteka postoji, radi se provjera postoje li svi potrebni konfiguracijski podaci unutar konfiguracijske datoteke. Konfiguracijski podaci koji se koriste su prikazani u tablici 2. Ako neki podatak nedostaje u datoteci ili ima pogrešnu vrijednost u odnosu na specifikaciju u tablici 2, javlja se poruka sa opisom koji podatak je problem i opisom razloga te poslužitelj prestaje s radom. Kada svi konfiguracijski podaci postoje u datoteci i kada su ispravni, poslužitelj čeka na zahtjeve za obradu. Ukoliko je proslijeđen drugi parametar umjesto čitanja informacije o portu iz datoteke koristi se navedeni port iz ulaznih parametara.

U tablici 2 definirani su nazivi svih parametara koji se koriste. Za svaki parametar napisani su kriteriji u kakvom obliku mora biti podatak te kratak opis čemu služi. Konfiguracijski podaci spremaju se u CSV datoteci u obliku “naziv\$vrjednost”, svaki u novi red. Vrijednost ne može imati znak \$.

Tablica 2. Pregled konfiguracijskih parametara

Naziv konfiguracije	Vrijednost	Opis
tajniKljucSesija	Veličina: 100 - 200 znakova. Dozvoljava: mala slova, brojke i specijalne znakove (!%\$)	Tajni ključ za sesiju
stranicaLimit	Broj od 10 do 50	Broj zapisa po stranici
tmdbApiKeyV3	Nema restrikcija	Sadrži API ključ v3 za pristup TMDB servisu oko 32 znaka
tmdbApiKeyV4	Nema restrikcija	Sadrži API ključ v4 za pristup TMDB servisu oko 211 znakova

Opis rada servisa

Servis predstavlja aplikativno programsko sučelje (API) prema bazi podataka (kao na slici 1). Servis nudi API za resurse na bazičnoj putanji **“/api”**. **Sva komunikacija je u JSON obliku osim ako nije drugačije definirano u tablici.** Kod slanja/primanja jednog podatka kao što je dohvat, ažuriranje ili dodavanje korisnika šalje se samo taj jedan objekt npr. `{"ime":"pero", "prezime":"kos", ...}` Kod slanja/primanja više podataka kao što je dohvat svih korisnika šalje se niz objekata ili prazan niz ako nema podataka npr. `[{"ime":"pero", "prezime":"kos"}, {"ime":"maro", "prezime":"marić"}]`

Kod svih zahtjeva mora postojati aktivna sesija za pristup servisu. Za dijelove aplikacije koje može koristiti korisnik gost treba posojati posebna putanja. Kod kreiranja servisa treba se voditi smjernice za implementaciju servisa koje su dane na predavanjima kolegija Razvoj web aplikacija ili kolegija Osnove web tehnologija.

Kod implementacije servisa statusni kodovi grešaka moraju odgovarati situaciji na primjer **405 “zabranjena metoda”** ili ako metodi smije pristupi samo administrator servis vraća statusni kod **403** sa opisom **“zabranjen pristup”** ili ako se pokuša dohvatiti resurs koji nije u specifikaciji tada servis vraća statusni kod **404** sa opisom **“nepostojeći resurs”**. Sve poruke grešaka se vraćaju unutar tijela odgovora i moraju imati smisla npr. za statusni kod 403 odgovor bi mogao izgledati ovako: `{"greska":"zabranjen pristup"}`

U svim zahtjevima gdje je uspješno odrađen zahtjev koji nešto dohvaća vraća se statusni kod **200** i **odgovarajući podaci**. U svim zahtjevima gdje je uspješno odrađen zahtjev koji ništa ne vraća npr. kreiranje, ažuriranje i brisanje vraća se statusni kod **201** i u tijelu `{"status":"uspjeh"}`

Student/ica treba napraviti tablicu koja predstavlja dokumentaciju implementiranog servisa za sve putanje i metode te ukratko opisati što servis vraća u slučaju pozitivne i negativne situacije. Ukratko iz tablice mora biti jasno kako se servis koristi.

Student koji ne napravi ovu specifikaciju ili koja ne odgovara stvarnom stanju gubi bodove iz servisnog dijela.

Potrebno je pripremiti datoteku sa curl naredbama za testiranje javnog dijela servisa. Testiranje napraviti za pozitivne, ali i negativne scenarije (npr. nedostaje parametar). Pokretanjem vašeg poslužitelja i kopiranjem naredbi one moraju uspješno raditi.